alura

ENCAPSULAMENTO

Praticando C# e OO

ESCOLA_ **PROGRAMAÇÃO**

Instrutora: Iasmin Araújo

```
Produto item1 = new Produto();
item1.preco = 100.00m;
item1.estoque = 10;

item1.AlterarPrecoComDesconto(15m);
item1.preco = item1.preco * (15/100);
```

Veja que, no cálculo manual, o preço está multiplicado por 15%, ao invés de aplicar o desconto fazendo a subtração.

ALTERANDO INFORMAÇÕES

- ✔ Atualmente, todas as informações da classe Produto podem ser alteradas a qualquer momento.
- ✔ Por exemplo, para fornecer desconto em um produto, podemos chamar o método `AlterarPrecoComDesconto()` ou fazer o cálculo manualmente.
- ✔ Poder fazer o cálculo manualmente abre brechas para erros inesperados. Isso pode gerar prejuízo para as organizações.

O ideal é que os atributos de uma classe não sejam acessíveis diretamente por quem for utilizá-la. Apenas as operações (métodos) que a classe executa devem estar visíveis.

Divulgamos **o que** a classe faz, não **como** ela faz. As implementações internas ficam ocultas.

Assim, os métodos encapsulam os atributos e os detalhes de implementação, caracterizando o princípio do **encapsulamento**.

REGRA GERAL PARA USAR ENCAPSULAMENTO

- ✔ Atributos **privados,** pois representam dados
- ✔ Métodos **públicos,** revelando as operações da classe

ESSES COMPORTAMENTOS SÃO GARANTIDOS VIA
MODIFICADORES DE ACESSO

PRIVATE

Modificador padrão para atributos e métodos. Visível apenas **dentro da própria classe**.

INTERNAL

Modificador padrão para classes. Visível para classes no **mesmo projeto**.
Também pode ser usado para atributos e métodos.

PUBLIC

Visível tanto em **classes diferentes** quanto em **projetos diferentes**. Para garantir o encapsulamento, devemos ter atenção a esse modificador.

```
Produto item1 = new Produto();
item1.nome = "Teclado";
item1.descricao = "Modelo compacto e silencioso," +
        " perfeito para produtividade diária.";
item1.preco = 80.00m;
item1.estoque = 15;
```

CRIANDO OBJETOS COM ENCAPSULAMENTO

- ✔ Ao deixar os atributos privados, o código de criação de um objeto passa a ter erros de compilação.
- ✔ Para garantir que os objetos sejam inicializados corretamente, garantindo a proteção dos atributos, usamos **métodos construtores**.

```
Produto item1 = new Produto();
```

Construtor padrão sendo utilizado.

```
class Produto
{
    // atributos da classe

    public Produto(){}
}
```

Um construtor padrão tem esse formato, declarado de forma implícita.

MÉTODOS CONSTRUTORES

- ✔ Os métodos construtores são responsáveis por inicializar os objetos na memória.
- ✔ Para chamar um construtor, utilizamos a palavra chave new.
- ✔ Por padrão, eles não recebem nenhum parâmetro.
- ✔ Se nenhum construtor está declarado explicitamente na classe, será usado um construtor padrão declarado implicitamente.

```
class Produto
{
  // atributos da classe
   public Produto(string nome, string
   descricao, decimal preco)
     {
         this.nome = nome;
         this.descricao = descricao;
         this.preco = preco;
         this.estoque = 0;
     }
}
```

Quando os parâmetros têm o nome igual ao dos atributos, usamos a palavra chave `this` para diferenciá-los.

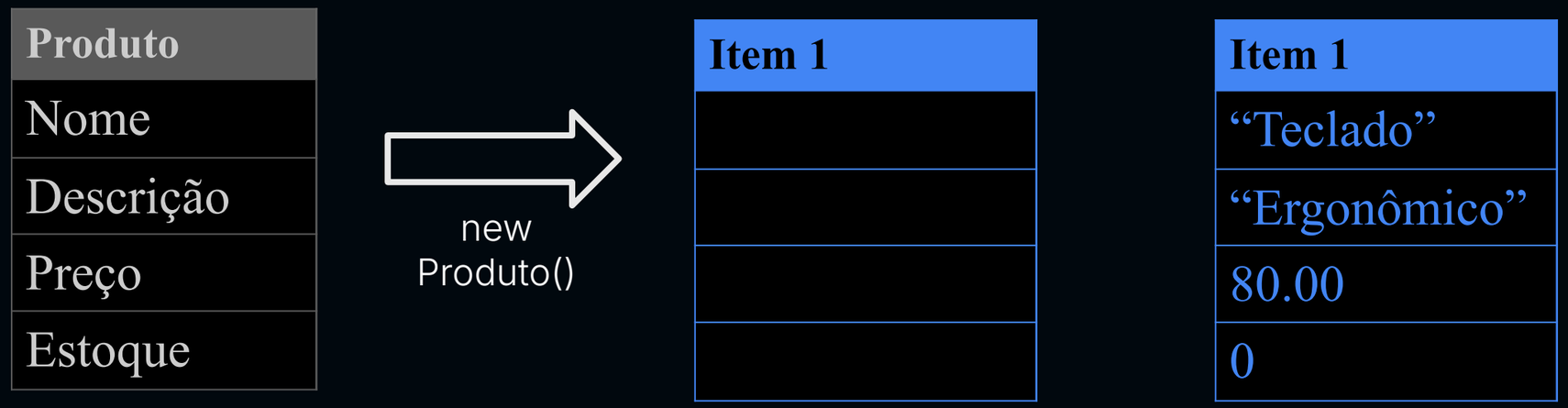
CONSTRUTORES PERSONALIZADOS

- ✔ Construtores podem ser personalizados, adicionando parâmetros e implementações no corpo do método.
- ✔ Com isso, ao inicializar o objeto, os valores dos atributos já podem ser atribuídos.

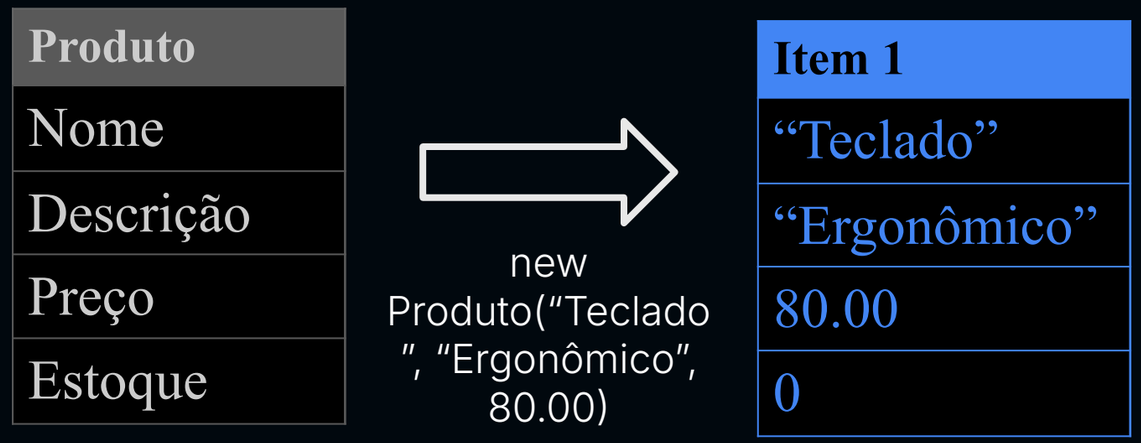
alura

CONFIRA A ESTRUTURA DE UM CONSTRUTOR PADRÃO E UM PERSONALIZADO.

CONSTRUTOR PADRÃO



CONSTRUTOR PERSONALIZADO



Passo 1

alura

PERCEBE A DIFERENÇA ENTRE O PADRÃO E O PERSONALIZADO?

Métodos construtores garantem o encapsulamento. Ao personalizar um construtor com parâmetros, garantimos que determinado atributo continuará privado e protegido.

Com isso, os valores dos atributos serão alterados somente na inicialização. Para modificações futuras, é necessário criar métodos específicos.

```
public string GetImagem()
{
    return imagem;
}

public void SetImagem(string imagem)
{
    if(imagem.Length > 0)
    {
        this.imagem = imagem;
    }
}
```

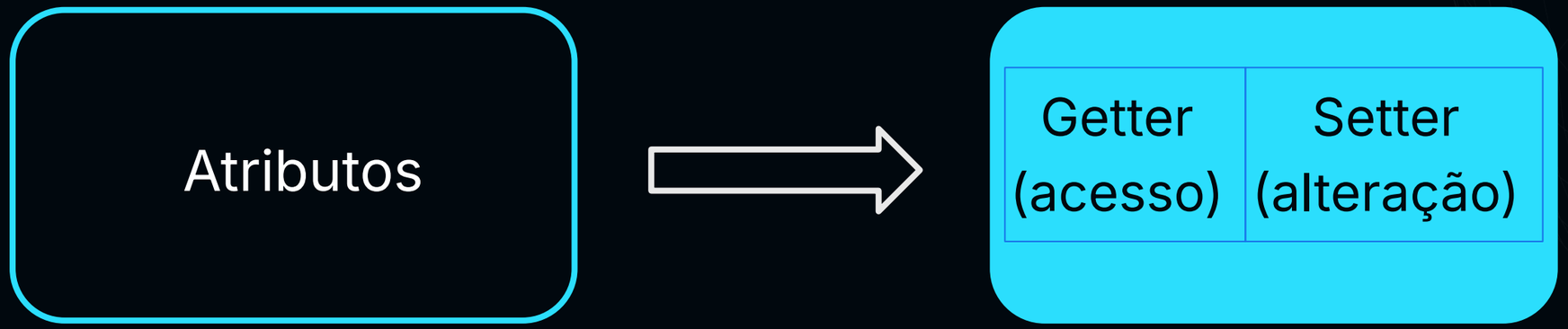
MÉTODOS COMUNS NA POO

- ✔ Acesso a dados: **getters**. Têm prefixo Get.
- ✔ Alteração de dados: **setters**. Têm prefixo Set.

Métodos getters e setters também garantem o encapsulamento.

Sem os getters, seria necessário tornar os atributos públicos sempre que precisássemos acessar seus valores, o que abriria margem para alterações indesejadas e violaria o encapsulamento.

Já ao utilizarmos os setters, podemos fazer validações antes de modificar os dados, garantindo maior proteção para as alterações.



```
Produto item1 = new Produto("Teclado",
"Modelo compacto e silencioso, perfeito
para produtividade diária.", 80.00m,
"Imagem");

item1.SetImagem("nova imagem");
Console.WriteLine(item1.GetImagem());

item1.Imagem = "nova imagem";
Console.WriteLine(item1.Imagem);
```

PROPRIEDADES

- ✔ Para trabalhar com getters e setters de uma maneira mais legível, o C# disponibiliza o recurso das propriedades (properties, em inglês).
- ✔ Ao lado, observamos o contraste entre os métodos tradicionais (getters e setters) e o uso de propriedades. O uso dos métodos convencionais é bem mais verboso!

```
private string imagem;
public string Imagem
  {
      get
      {
          return imagem;
      }
      set
      {
          if (value.Length > 0)
          {
              this.imagem = value;
          }
      }
  }
```

Ao invés de trabalhar com parâmetros, o valor a ser alterado no atributo é chamado de **value.**

CRIANDO PROPRIEDADES

- Para criar uma propriedade, iremos usar:
  - um modificador de acesso (geralmente public);
  - o tipo de dado que aquela propriedade irá guardar (é o mesmo do atributo);
  - o nome da propriedade (geralmente é o nome do atributo no formato PascalCase).
- Dentro das chaves, declaramos dois blocos get e set, que terão comportamento igual ao do getter e setter do atributo.

```
class Produto
{
    public string Nome { get; }
    public string Descricao { get; }
    public decimal Preco { get; private
set; }
}
```

PROPRIEDADES AUTOIMPLEMENTADAS

- ✔ Quando não há lógica adicional nos métodos get e set, é possível utilizar apenas palavras-chave. Assim, o compilador irá gerar implicitamente um atributo privado para armazenar o valor da propriedade.
- ✔ Isso é o suficiente para que o get retorne o valor do atributo e o set faça uma alteração simples, sem validações.
- ✔ Cuidado ao declarar gets e sets para todas as propriedades! Isso pode ferir o princípio do encapsulamento.

```
public Produto(string nome, string
descricao, decimal preco, string imagem)
{
        this.Nome = nome;
        this.Descricao = descricao;
        this.Preco = preco;
        this.Imagem = imagem;
}

public void AlterarPrecoComDesconto(decimal
desconto)
{
        Preco = Preco * (1 - desconto/100);
}
```

PROPRIEDADES

- ✔ Pode ser vantajoso utilizar as propriedades dentro do construtor. Assim, para todas as atribuições, serão feitas as validações do set (se estiverem implementadas).
- ✔ Em métodos fora do construtor, qualquer alteração em uma propriedade exige que ela possua um setter.
- ✔ Para evitar trabalhar com sets públicos e indesejados, podemos utilizar `private set`. Ele indica que a propriedade pode ser alterada apenas dentro da classe.

alura

Compartilhe um resumo de seus novos conhecimentos em suas redes sociais.

#aprendizadoalura